Baseline vs Online Join vs Stage 1 vs Stage 2

How each implementation works and what improved

Ann Remizova

June 9, 2026

Goal

- The task is hybrid movie search over structured columns and free-text reviews.
- SUQL gives us SQL plus text functions such as `answer(review, question)` and `summary(review)`.
- The expensive part is repeatedly asking an LLM whether each review satisfies the semantic predicate.
- The optimization question: reduce expensive LLM calls without changing the high-level SUQL interface.

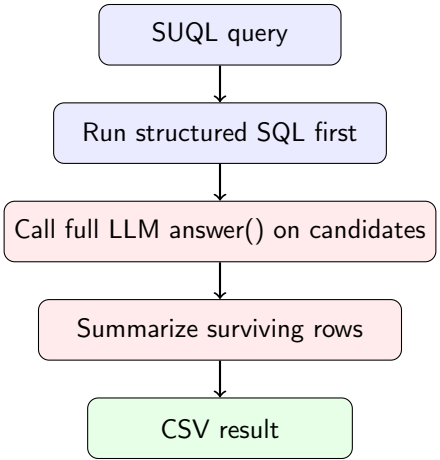
Common Query Shape

Example SUQL query

```
SELECT movie_id, title, year, runtime, director, genres,
 summary(review) AS review_summary
FROM movies
WHERE year >= 1990 AND genres LIKE '%Comedy%'
 AND answer(review, 'Does the reviewer say it is funny?')  = 'Yes'
LIMIT 10;
```

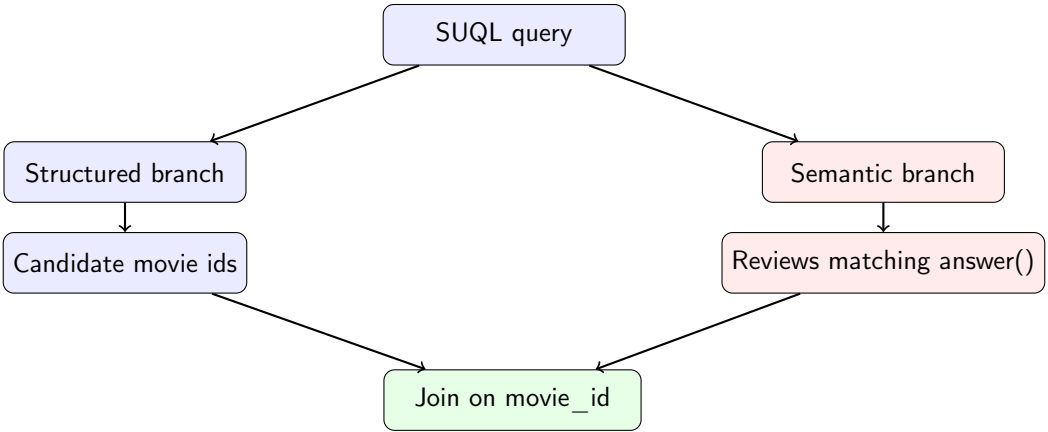
- Structured predicates: `year`, `runtime`, `genres`, `director`.
- Unstructured predicate: `answer(review, ...)`.
- Output enrichment: `summary(review)` for rows that survive filtering.

Baseline: SUQL Execution



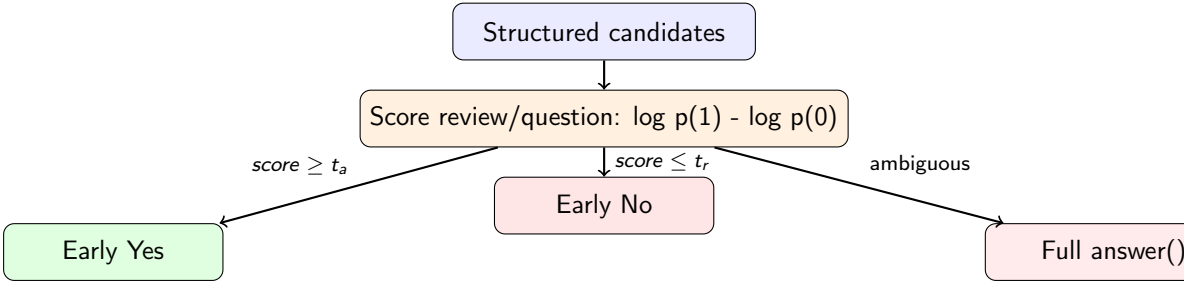
- This follows the SUQL idea: combine structured database filtering with LLM-backed text operators.
- Strength: structured predicates reduce the number of reviews sent to the LLM.

Online Join: Alternative Execution Order



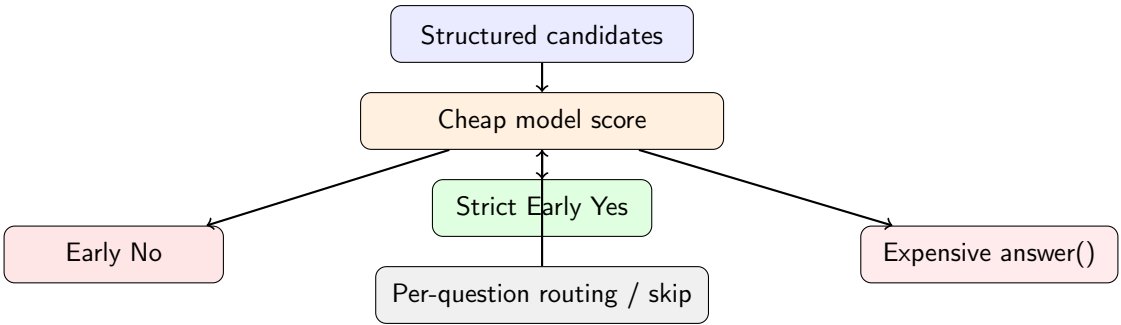
- Structured and semantic retrieval are independent branches.
- In the current implementation, the semantic branch scans the whole review sample.
- It is conceptually clean, but it loses the baseline's most important pruning step.

Stage 1: Calibrated Early Decisions



- Inspired by STRETTO-style cascades: use confidence thresholds to avoid some full LLM calls.
- Thresholds are calibrated per question from labelled examples.
- Limitation in this implementation: the scoring call still uses the expensive model, so it can save full calls but add prompt overhead.

Stage 2: Cheap-to-Expensive Cascade



- Uses different models: cheap scorer `gemma2:2b`; expensive answer model `phi4-mini`.
- Cheap early accept is made strict to avoid false positives.
- Latest v7 routes cheap calls only to questions where previous runs showed benefit.

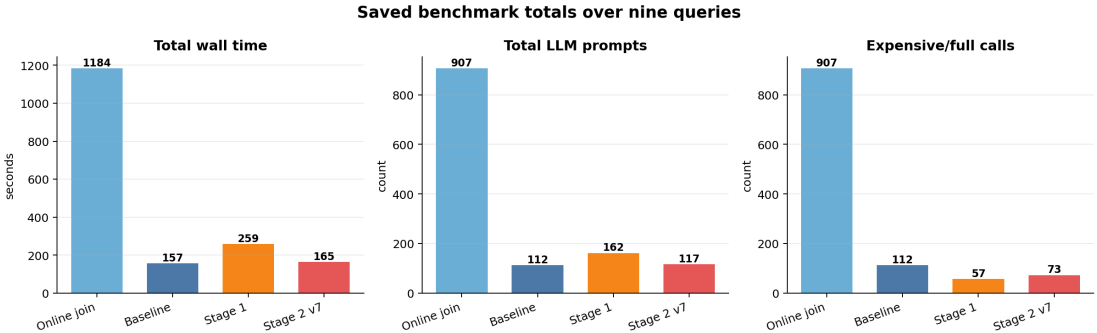
Evolution of the Implementation

Version	Main idea	Practical result
Baseline	Structured-first SUQL, full LLM on candidates	Good pruning, but every semantic candidate is expensive
Online join	Run structured and semantic branches independently, then join	Very expensive when semantic branch scans all reviews
Stage 1	Calibrated early accept/reject using log-odds thresholds	Fewer full calls, but scoring itself adds expensive prompts
Stage 2	Cheap model before expensive model, with routing and stricter accept	Large reduction in expensive calls; best optimized version so far

Benchmark Provenance

- The query set contains nine raw SUQL movie-review queries.
- Baseline and Stage 2 v7 numbers are from `experiment_v7_gemma2_routed_winners`.
- Online join numbers are from `online_join_vs_stage2_v7`, run on the same Stage 2 sample.
- Stage 1 numbers are from `stage1_medium_different_questions_fixed_scorer`; same query design, separate saved run.
- Therefore, online join vs Stage 2 v7 is the strictest direct comparison; Stage 1 is included as a design and trend comparison.

Total Runtime and Calls



Aggregate Metrics

System	Wall s	Engine s	Prompts	Expensive/full	Result rows
Online join	1184.45	1132.63	907	907	7
Baseline	157.40	134.89	112	112	7
Stage 1	259.23	228.46	162	57	18
Stage 2 v7	164.56	132.98	117	73	11

- Online join is dominated by full semantic scans.
- Stage 1 cuts full calls from 112 to 57 in its run, but total prompts increase to 162.
- Stage 2 v7 keeps prompt count close to baseline while reducing expensive calls from 112 to 73.

Direct Comparison: Online Join vs Stage 2 v7

Metric	Online join	Stage 2 v7	Change
Wall time	1184.45 s	164.56 s	7.2x faster
Engine time	1132.63 s	132.98 s	8.5x faster
LLM prompts	907	117	7.8x fewer
Expensive calls	907	73	12.4x fewer

- Online join evaluates about 100 reviews per query with the expensive model.
- Stage 2 first applies structured predicates, then runs cheap/exensive semantic checks only on candidates.
- This is the clearest win in the experiments.

Stage 1: What Improved and What Did Not

Improved

- Added calibrated thresholds.
- Avoided many full answer generations.
- Made answer decisions measurable: score calls, early accepts, early rejects.

Problem

- The score call was not cheap enough.
- Total prompts increased.
- Early accepts sometimes changed result rows too much.

Observed

Stage 1 reduced full calls, but wall time rose from 179.55 s to 259.23 s in the saved Stage 1 benchmark.

Stage 2: Fixes Added After Stage 1

- **Different model roles:** cheap scorer uses a smaller Ollama model; expensive path keeps `phi4-mini`.
- **Strict accept floor:** cheap label-only "Yes" is not enough for early acceptance.
- **Adaptive skip:** if cheap decisions are too rare, stop probing cheaply for that question.
- **Question routing:** latest v7 only uses cheap scoring on questions where it previously helped.
- **Model tracking:** metrics record cheap model, expensive model, cheap calls, expensive calls, skipped, disabled, and per-question usage.

Why Stage 2 v7 Is the Best Stage 2 Version

Stage 2 run	Wall s	Prompts	Expensive calls	Cheap calls
v2 naive cheap	294.26	186	2	105
v3 strict accept	281.78	217	99	105
v4 adaptive skip	223.60	159	101	45
v5 gemma2	270.83	132	23	100
v6 routed	183.04	124	32	83
v7 routed winners	164.56	117	73	33

- v7 is the fastest Stage 2 run.
- It trades some expensive-call savings for lower overhead.
- It uses cheap scoring only where cheap scoring had a high payoff.

Relationship to SUQL

- All versions preserve the SUQL programming model: SQL plus free-text functions.
- Baseline follows the natural SUQL execution order for this dataset: structured filtering before `answer()`.
- Online join explores a different physical plan: independent structured and semantic branches followed by a join.
- Stage 1 and Stage 2 do not change SUQL syntax; they change the physical implementation of `answer()`.

Relationship to STRETTO

- STRETTO's relevant idea here is cascade execution: use cheaper evidence first, escalate only when needed.
- Stage 1 implements the cascade idea with calibrated log-odds thresholds.
- Stage 2 makes the cascade more faithful in practice by separating cheap and expensive models.
- The experiments show the key lesson: a cascade helps only if the first stage is both accurate enough and genuinely cheap.

Main Takeaways

1. **Best runtime design:** Stage 2 v7.
2. **Worst runtime design:** online join, because it scans reviews semantically before structured pruning.
3. **Stage 1 lesson:** fewer full calls do not guarantee speed if the scoring call is also expensive.
4. **Stage 2 lesson:** routing matters; cheap model calls should be used only where they save enough expensive calls.
5. **Current limit:** Stage 2 v7 is close to baseline wall time, but not a large win over baseline yet.

Next Improvements

- Batch cheap scoring calls where Ollama/backend support allows it.
- Add a deterministic no-summary benchmark mode to isolate filtering cost from output summarization.
- Calibrate thresholds with more labelled examples per question.
- Consider KV-cache-aware or prefix-reuse prompting only if the backend exposes reusable context safely.
- Compare final rows against labelled relevance to measure quality, not only runtime.

Artifacts

- Slides: `docs/presentations/system_comparison_slides.tex`
- Summary figure: `docs/assets/four_systems_summary.png`
- Online join vs Stage 2 metrics: `Stage_2/benchmarks/online_join_vs_stage2_v7/metrics.csv`
- Stage 2 v7 metrics: `Stage_2/benchmarks/experiment_v7_gemma2_routed_winners/metrics.csv`
- Stage 1 metrics:
  `Stage_1/benchmarks/stage1_medium_different_questions_fixed_scorer/metrics.csv`